

XBOT EV

TASK 3

Automation: Workflow Design

WhatsApp Conversation Architecture • Backend/API Design • Fallbacks • Security • Governance



Proposed design for the XBOT EV hiring assessment

1. Executive Overview

This submission presents a proposed WhatsApp automation design for XBOT EV covering the two required customer intents: checking the official price of a specified bike model and booking a test ride.

The design combines conversational AI with controlled backend validation, server-side persistence, fallback handling, human handoff, security, observability, and governance.

Design guardrail

AI understands the customer; the backend validates business rules; PostgreSQL stores the source of truth. This is an implementation-oriented design and does not claim that the bot, APIs, database, or integrations are already live in production.

Assessment scope at a glance

Area	Proposed design response
Primary channel	WhatsApp via Twilio WhatsApp API
Intent 1	Official bike price enquiry
Intent 2	Test-ride booking with explicit confirmation
Business logic	FastAPI + Python
Source of truth	PostgreSQL for catalog, slots, bookings, and status
Exception handling	Guided fallback plus human handoff

2. Channel Decision — WhatsApp vs Voice

WhatsApp is selected for this assessment because the two required intents benefit from structured text, visible confirmations, persistent conversation context, and straightforward recovery when information is missing or invalid.

Evaluation criterion	WhatsApp	Voice
Structured data capture	Strong	Moderate
Visible booking confirmation	Strong	Limited
Error correction / retry	Straightforward	More complex
Demo clarity for assessment	Strong	Moderate
Additional speech stack	Not required	ASR + TTS required
Selected for this design	Yes	No

Decision

Use WhatsApp as the proposed customer channel, with Twilio WhatsApp API as the messaging gateway. Voice remains a valid future channel, but it adds speech-recognition, text-to-speech, latency, interruption, and testing complexity that is unnecessary for this assessment scope.

3. System Architecture & Master Flow Diagram

The architecture separates the customer-facing conversation from backend business logic and persistent data. The controlled path is: Customer → WhatsApp → Twilio webhook → FastAPI backend → intent routing → business logic → PostgreSQL / optional integrations → response, logging, or human handoff.

XBOT EV — WhatsApp Conversation Workflow

Price Enquiry + Test-Ride Booking + Fallback / Human Handoff

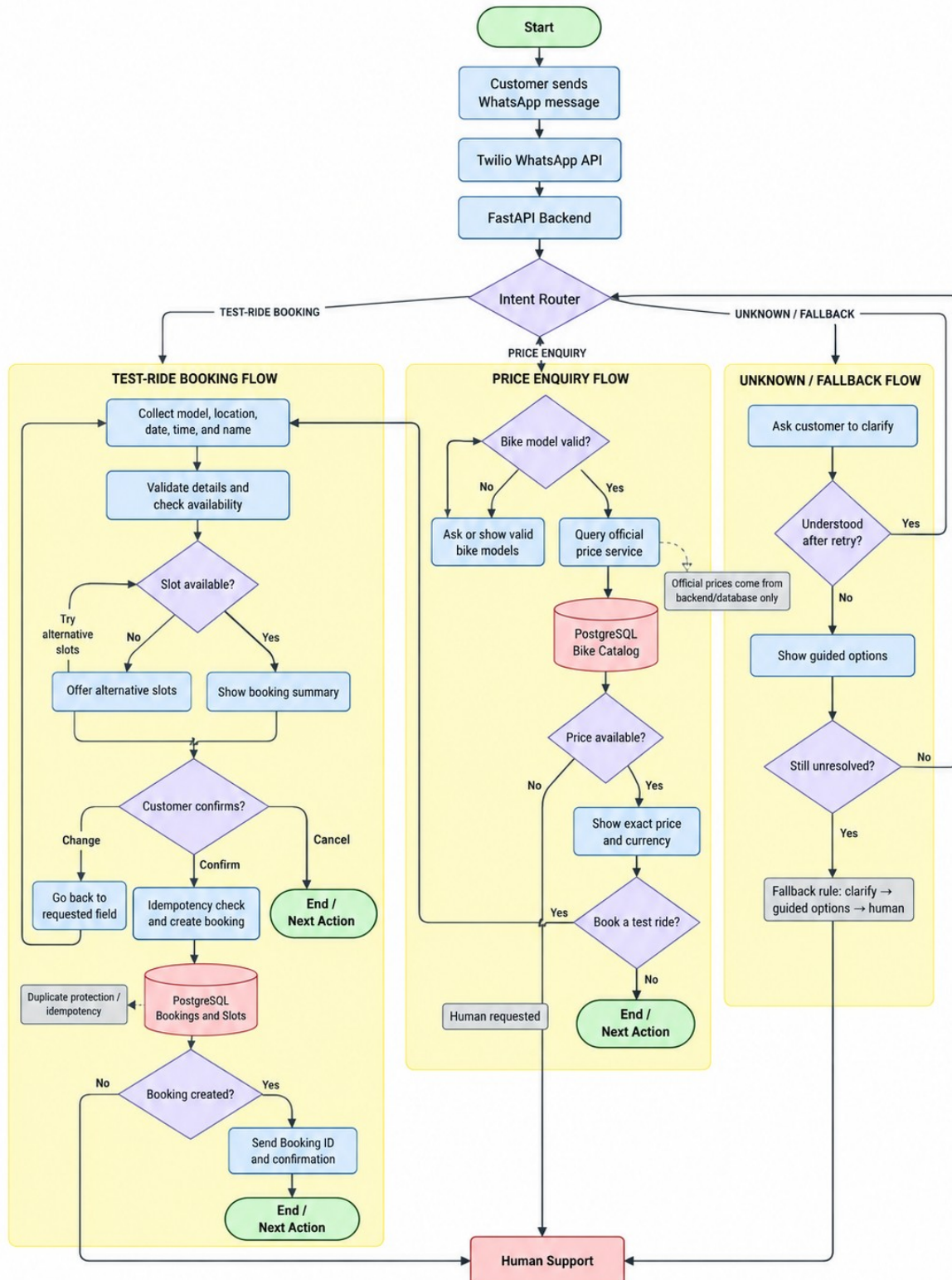


Figure 1. XBOT EV WhatsApp master conversation workflow — price enquiry, test-ride booking, fallback handling, and human handoff.

4. Price Enquiry Intent — Detailed Logic

Purpose

Allow a customer to request the official price of a specific XBOT EV model. AI can interpret the wording, but it must never invent or guess a price.

Step	Logic
1. Receive enquiry	Customer asks for the price of an XBOT EV bike model.
2. Extract model	Identify the model name from the customer message.
3. Validate model	Backend checks the model against the approved XBOT EV bike catalog.
4. Missing / invalid model	Ask the customer to select or provide a valid model; do not guess.
5. Retrieve official price	Backend reads the price from the approved pricing source / PostgreSQL catalog.
6. Price unavailable	Do not invent a value; offer human support.
7. Price available	Show the exact official price and currency.
8. Next action	End the interaction or continue into the test-ride booking flow.

Source-of-truth rule

Official pricing must come from a controlled backend data source. The language model is not an authoritative pricing system.

5. Test-Ride Booking Intent — Detailed Logic

Purpose

Collect the required booking details, validate them, check availability, show a summary, and require explicit customer confirmation before any booking is created.

Step	Logic
1. Receive request	Customer asks to book a test ride.
2. Collect details	Capture selected model, preferred location, date, time, and customer name.
3. Validate model	Confirm the selected XBOT EV model is valid.
4. Validate date	Require a valid future date.
5. Validate time	Require a supported operating window / available booking window.
6. Check availability	Query the booking source of truth for the requested slot.
7. Slot unavailable	Offer alternative available slots.
8. Show summary	Display model, location, date, and time before creation.
9. Customer decision	Allow Confirm, Change, or Cancel.
10. Confirm	Run an idempotency / duplicate check, then create the booking.
11. Change	Return to the requested field, update it, and show the summary again.
12. Cancel	Stop without creating a booking.
13. Booking success	Return the confirmed booking ID and confirmation only after backend success.
14. Booking failure	Do not claim success; offer retry or human support.

Persistence & duplicate-protection rule

The booking is created through the backend and stored server-side in PostgreSQL. Repeated confirmations or network retries must return the existing booking result rather than create a second booking.

6. Fallback & Edge-Case Handling

The fallback design prevents guessing, endless loops, duplicate bookings, and false success messages. The system should fail safely and escalate when automation should stop.

Scenario	Safe system response	Why it matters
Bike model missing	Ask which XBOT EV model the customer means.	Prevents guessing.
Invalid bike model	Show valid XBOT EV models and ask again.	Avoids false product information.
Price unavailable	Do not invent a price; offer human support.	Protects pricing accuracy.
Past / invalid date	Ask for a valid future date.	Prevents invalid bookings.
Invalid / unavailable time	Show supported hours or alternative available slots.	Keeps bookings realistic.
Customer cancels	Stop without creating a final booking.	Prevents accidental bookings.
Duplicate confirmation	Return the existing booking result.	Prevents duplicate bookings.
Customer changes details	Update the requested field and show the summary again.	Allows safe corrections.
Booking creation fails	Do not say confirmed; offer retry or human support.	Keeps status truthful.
Booking succeeds; message fails	Keep the booking; retry only the notification.	Prevents a second booking.
Repeated misunderstanding	Clarify, show guided options, then escalate if unresolved.	Prevents endless fallback loops.
Customer asks for human	Transfer with conversation summary and booking context.	Avoids restarting from zero.

Fallback escalation rule

1st misunderstanding	2nd misunderstanding	3rd unresolved attempt
Ask the customer to clarify.	Show guided options: Check bike price • Book a test ride • Talk to support	Transfer to human support with current conversation context.
Loop-prevention rule The system must never remain in an endless “I didn’t understand” loop.		

7. Proposed Tool / API Stack

The stack below separates messaging, AI understanding, business logic, persistence, automation, observability, security, and human support. All components are proposed for this assessment; no live integration is claimed unless explicitly demonstrated.

Layer	Proposed tool	Role / justification	Guardrail
Channel	Twilio WhatsApp API	Receives WhatsApp messages and sends replies through webhooks.	Verify webhook signatures; use HTTPS.
Backend	FastAPI + Python	Owns validation, business logic, API requests, and booking operations.	Business decisions stay server-side.
Database	PostgreSQL	Durable source of truth for models, prices, slots, bookings, and status.	No browser-only storage or AI-generated official values.
AI / NLP	OpenAI API with Structured Outputs	Classifies intent and extracts structured fields from natural language.	Must not invent prices, availability, booking IDs, or success.
Workflow automation	n8n	Handles secondary automation such as dealer alerts, CRM updates, email, and follow-up.	Not the booking source of truth.
Availability	Internal slot system (PostgreSQL); optional Google Calendar API	Checks availability; optionally creates a calendar event after confirmed booking.	Calendar is downstream / optional, not authoritative unless designed that way.
Observability	Structured application logs	Records request/conversation IDs, intent, tools, result, errors, latency, and timestamps.	Avoid unnecessary PII in logs.
Secrets	Environment variables or secret manager	Protects API keys, database credentials, and webhook secrets.	Never hard-code or expose secrets publicly.
Human handoff	Internal CRM / support queue	Transfers unresolved or customer-requested conversations with context.	Pass summary, model, current step, and escalation reason.
Architecture principle n8n and AI can assist the workflow, but confirmed booking state and official business data remain controlled by the backend and PostgreSQL.			

8. Security, Privacy & Governance

Security and governance are part of the design because the workflow handles customer information, backend APIs, booking data, third-party integrations, and AI-driven language understanding.

Control	Implementation	Why it matters
HTTPS & secure webhooks	Use HTTPS for API traffic and verify incoming webhook signatures.	Reduces fake-request and interception risk.
API keys & secrets	Store keys, database passwords, and webhook secrets server-side in environment variables or a secret manager.	Prevents credential exposure.
Customer data / PII	Collect only information needed for the test ride; store securely and restrict access.	Reduces unnecessary privacy exposure.
Least privilege	Give staff and system components only the permissions required for their responsibilities.	Limits the impact of misuse.
Input validation & rate limiting	Reject malformed requests and apply sensible rate limits.	Reduces bad data, overload, and abuse.
Data retention & safe logging	Keep only required customer/operational data and define a production retention policy.	Avoids indefinite unnecessary retention.
AI governance	Allow AI to classify/extract, while official prices, availability, IDs, and booking status come from validated services.	Reduces hallucination and unsafe automated actions.
Auditability & change control	Record key events and review important workflow/business-rule changes before production use.	Supports QA, investigation, and traceability.
Governance summary AI understands language. Backend services enforce business rules. PostgreSQL stores authoritative state. Human support receives cases that should leave automation.		

9. Short Case Study

9.1 Business Problem

XBOT EV customers may contact the company to ask for bike prices or arrange a test ride. If these requests are handled manually, customers may wait for responses, staff may repeat the same answers, and booking details can become inconsistent.

9.2 Proposed Solution

The proposed solution is a WhatsApp-based XBOT EV bot supporting two primary intents: provide the official price of a selected bike model, and book a test ride. AI is used to understand customer language and intent; official prices, slot availability, booking IDs, and booking status come from controlled backend services and server-side data sources.

9.3 Workflow Design

For a price request, the bot identifies and validates the model, then retrieves the official price from the backend source of truth. For a test-ride request, it collects required details, validates date and time, checks slot availability, shows a summary, and asks for explicit confirmation before creation. The customer can Change, Cancel, or request Human Support. Duplicate protection prevents repeated confirmations or network retries from creating multiple bookings.

9.4 Reliability, Safety & Governance

The design separates AI understanding from deterministic business logic. Fallback handling, human handoff, secure secret management, minimal customer-data collection, logging, auditability, and controlled backend actions reduce hallucination risk and keep booking outcomes traceable.

9.5 Business Value & Expected Benefits

- Faster responses to common price enquiries.
- More consistent use of official pricing data.
- Fewer incomplete or invalid test-ride booking attempts.
- Reduced duplicate-booking risk through idempotency controls.
- Better handoff to human support when automation should stop.
- Clearer operational records for support, QA, and troubleshooting.

Evidence statement

These are expected benefits of the proposed design, not claimed production results.

Assessment Deliverable Map

Required deliverable	Where it appears	Status
Flow diagram covering both intents + fallback / edge cases	Sections 3, 4, 5, and 6	Covered
Proposed tool / API stack with justification	Section 7	Covered
Short case study	Section 9	Covered